

Graphs

Agenda:

- Introduction & Definition
- Graph modeling
- Types of Graphs
- Operations on graphs
- Representing Graphs
- Applications

Introduction to Graphs

- **Definition:** A **directed graph** $G = (V, E)$ consists a set of vertices V and a set of arcs E .
- The vertices are also called nodes or points. The arcs are also called directed edges or directed lines.
- An arc is an ordered pair of vertices (v, w) . v is called the tail and w the head of the arc. The arc (v, w) is often expressed by $v \rightarrow w$.
- We say that $v \rightarrow w$ is from v to w and that w is adjacent to v .

Introduction to Graphs

➤ **Definition:** An **undirected graph (simple graph)**

$G = (V, E)$ consists of a finite set of vertices V and a set of edges E .

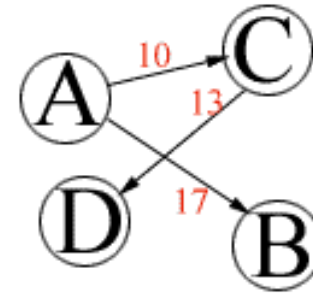
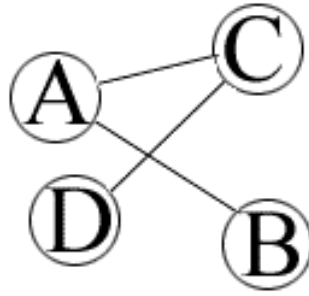
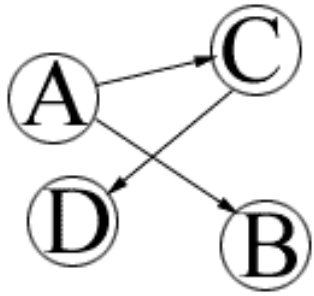
➤ Each edge in E is an unordered pair of vertices. If (v, w) is an undirected edge, then $(v, w) = (w, v)$

➤ We say that the edge (v, w) is incident upon vertices v and w .

➤ A path is a sequence of vertices $v_1, v_2, \dots, v_n$ such that (v_i, v_{i+1}) is an edge for $1 \leq i \leq n$.

➤ A path is simple if all the vertices on the path are distinct with the exception of v_1 and v_n which may be the same. The length of the path is $n-1$ i.e. the number of edges along the path.

Introduction to Graphs



Directed Graph:- Vertices: A, B, C and D Edges: AB, AC, CD

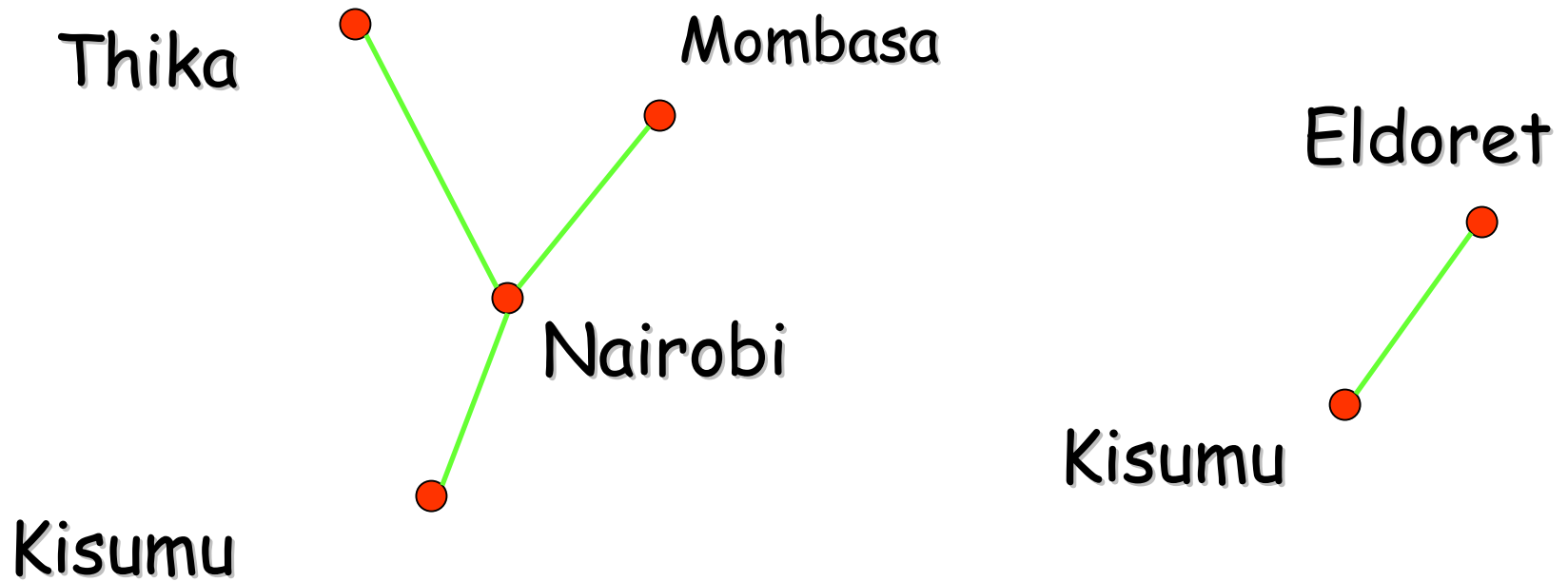
Undirected Graph:- Vertices: A, B, C, D Edges: AB, AC, CD

Weighted Graph:- Vertices: A, B, C, D Weighted Edges: AC-10, AB-17, CD-13

Graph Models

Example I: How can we represent a network of (bi-directional) railways connecting a set of cities?

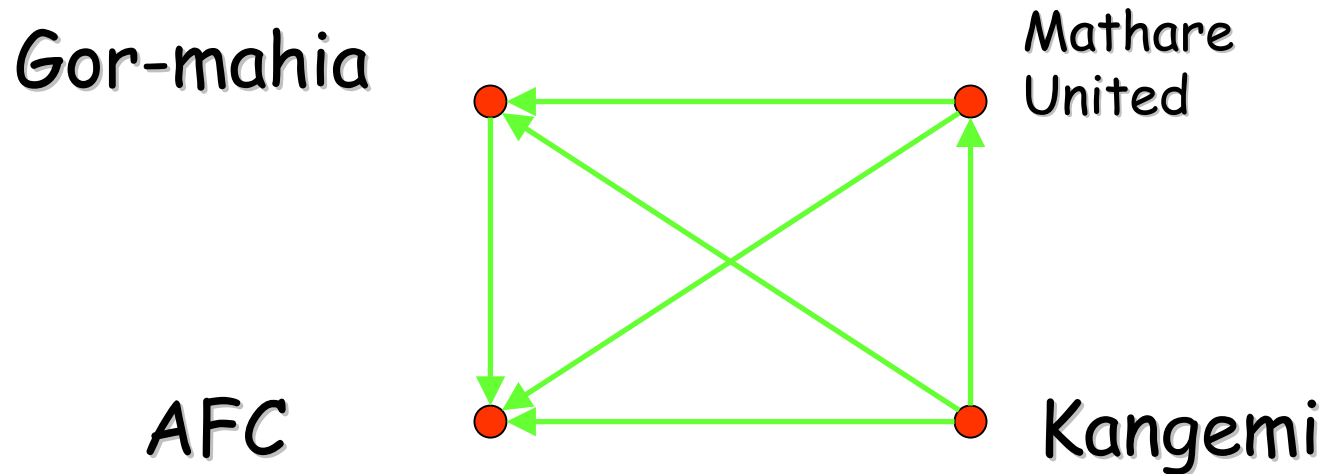
We should use a **simple graph** with an edge $\{a, b\}$ indicating a direct train connection between cities a and b .



Graph Models

Example II: In a round-robin tournament, each team plays against each other team exactly once. How can we represent the results of the tournament (which team beats which other team)?

We should use a **directed graph** with an edge (a, b) indicating that team a beats team b .



Graph Terminology

Definition: The **degree** of a vertex in an undirected graph is the number of edges incident on it, except that a loop at a vertex contributes twice to the degree of that vertex.

In other words, you can determine the degree of a vertex in a displayed graph by **counting the lines** that touch it.

The degree of the vertex v is denoted by **$\deg(v)$** .

Graph Terminology

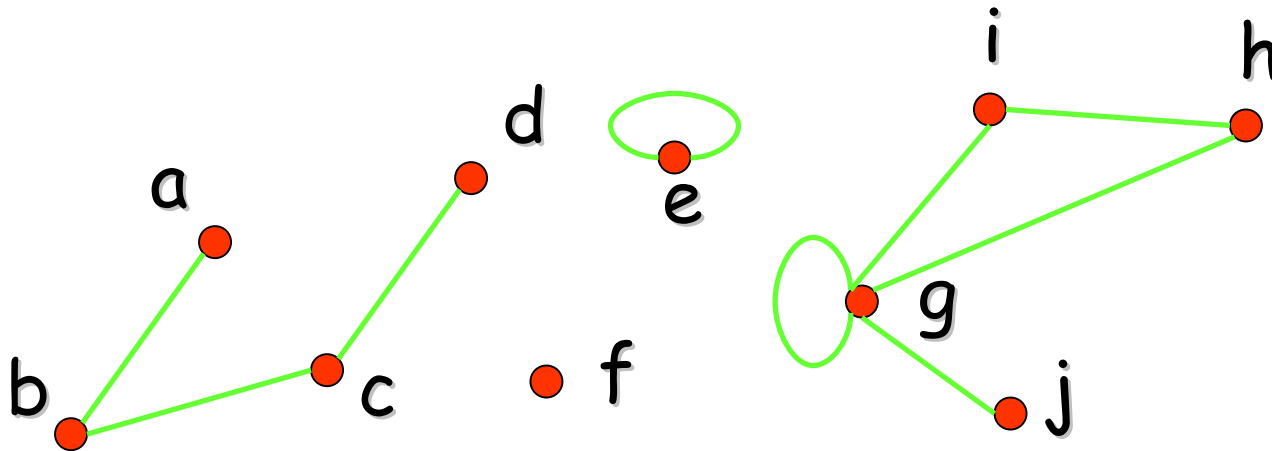
A vertex of degree 0 is called **isolated**, since it is not adjacent to any vertex.

Note: A vertex with a **loop** has at least degree 2 and, by definition, is **not isolated**, even if it is not adjacent to any **other** vertex.

A vertex of degree 1 is called **pendant**. It is adjacent to exactly one other vertex.

Graph Terminology

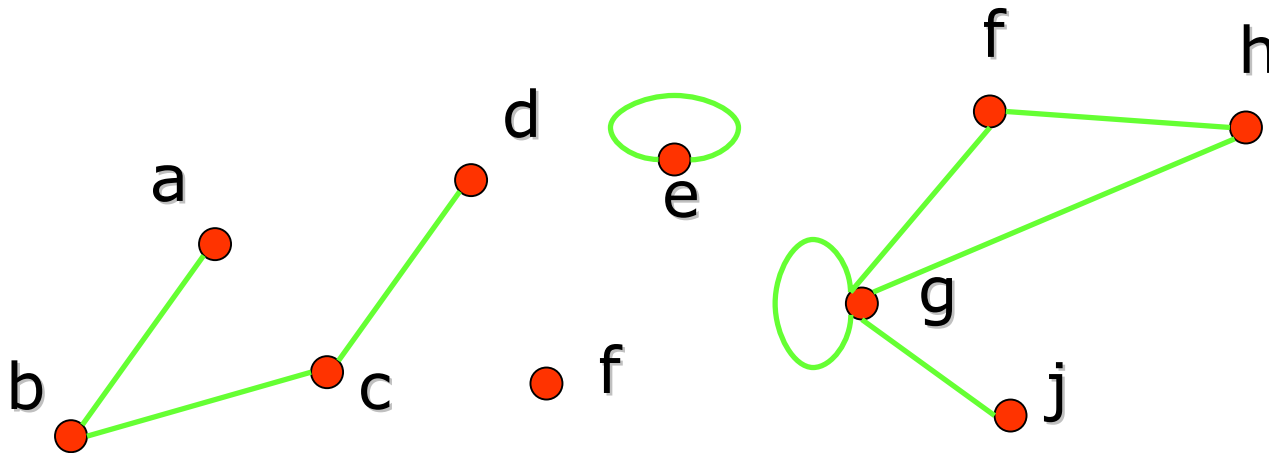
Example: Which vertices in the following graph are isolated, which are pendants, and what is the maximum degree? What type of graph is it?



Solution: Vertex f is isolated, and vertices a, d and j are pendants. The maximum degree is $\deg(g) = 5$. This graph is a simple graph

Graph Terminology

Let us look at the same graph again and determine the number of its edges and the sum of the degrees of all its vertices:



Result: There are 9 edges, and the sum of all degrees is 18. This is easy to explain: Each new edge increases the sum of degrees by exactly two.

Graph Terminology

Definition: In a graph with directed edges, the **in-degree** of a vertex v , denoted by $\deg^-(v)$, is the number of edges with v as their **terminal (head) vertex**.

The **out-degree** of v , denoted by $\deg^+(v)$, is the number of edges with v as their initial (tail) vertex.

Question: How does adding a loop to a vertex change the in-degree and out-degree of that vertex?

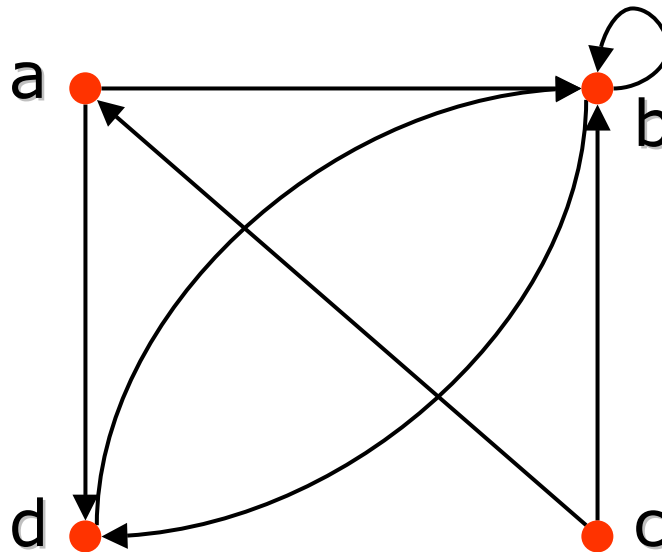
Answer: It increases both the in-degree and the out-degree by one.

Graph Terminology

Example: What are the in-degrees and out-degrees of the vertices a, b, c, d in this graph:

$$\deg^-(a) = 1$$
$$\deg^+(a) = 2$$

$$\deg^-(d) = 2$$
$$\deg^+(d) = 1$$



$$\deg^-(b) = 4$$
$$\deg^+(b) = 2$$

$$\deg^-(c) = 0$$
$$\deg^+(c) = 2$$

Special Graphs

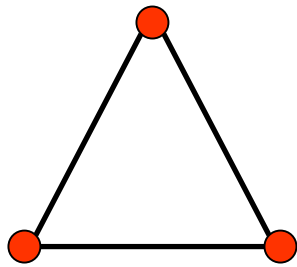
Definition: The **complete graph** on n vertices, denoted by K_n , is the simple graph that contains exactly one edge between each pair of distinct vertices.



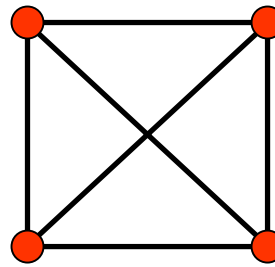
K_1



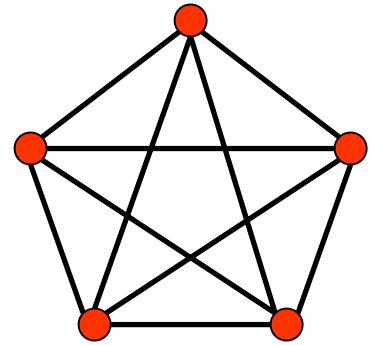
K_2



K_3



K_4

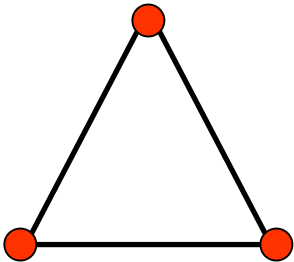


K_5

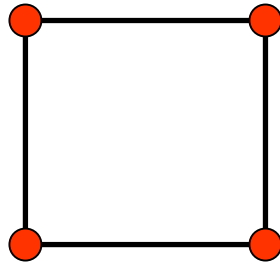
K_1 is complete because it does not contain any pair of distinct vertices.

Special Graphs

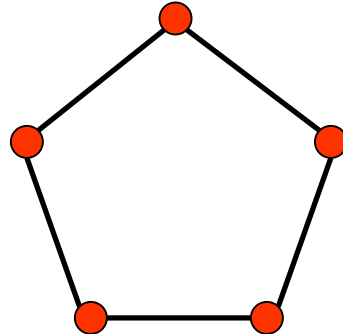
Definition: The **cycle** C_n , $n \geq 3$, consists of n vertices $v_1, v_2, \dots, v_n$ and edges $\{v_1, v_2\}, \{v_2, v_3\}, \dots, \{v_{n-1}, v_n\}, \{v_n, v_1\}$.



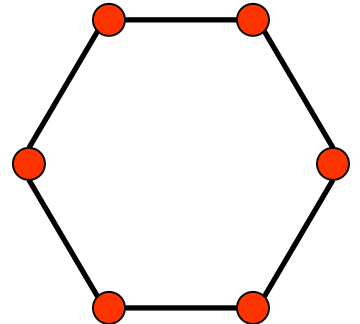
C_3



C_4



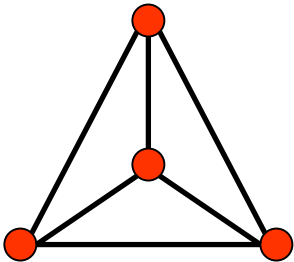
C_5



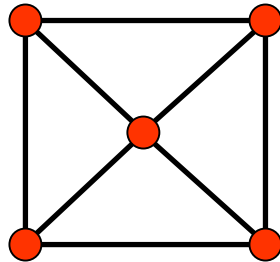
C_6

Special Graphs

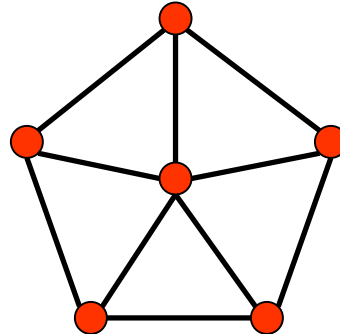
Definition: We obtain the **wheel** W_n when we add an additional vertex to the cycle C_n , for $n \geq 3$, and connect this new vertex to each of the n vertices in C_n by adding new edges.



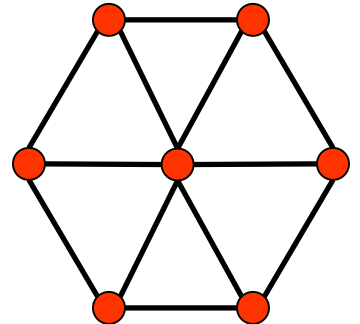
W_3



W_4



W_5

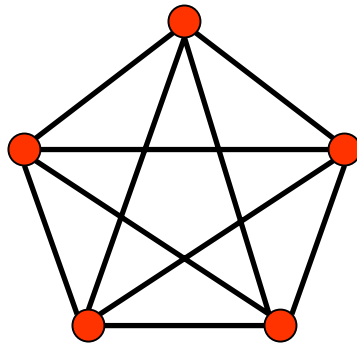


W_6

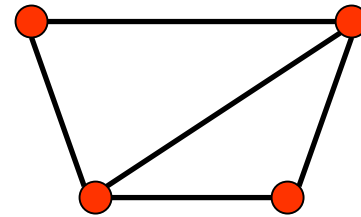
Operations on Graphs

Definition: A **subgraph** of a graph $G = (V, E)$ is a graph $G' = (V', E')$ where $V' \subseteq V$ and $E' \subseteq E$.

Example:

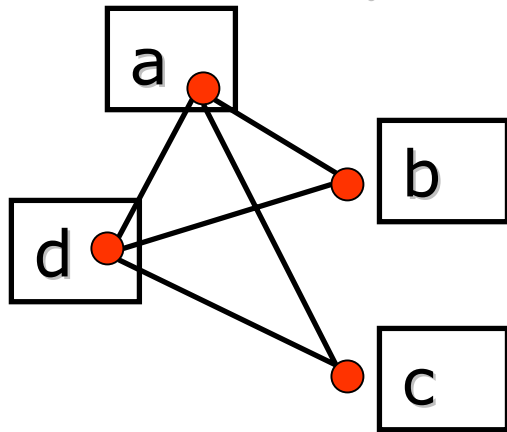


K_5

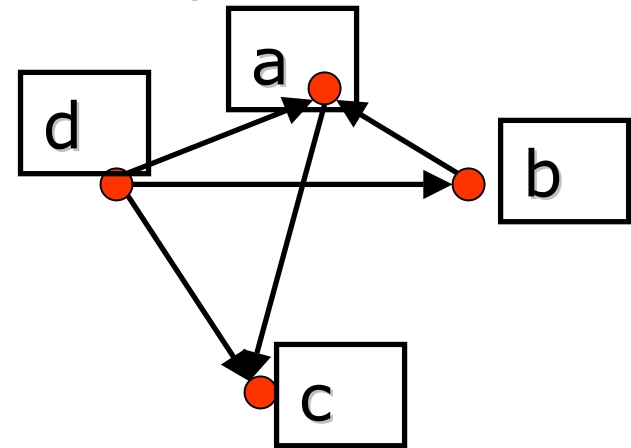


subgraph of K_5

Representing Graphs



Vertex	Adjacent Vertices
a	b, c, d
b	a, d
c	a, d
d	a, b, c



Initial Vertex	Terminal Vertices
a	c
b	a
c	
d	a, b, c

Representing Graphs

Definition: Let $G = (V, E)$ be a simple graph with $|V| = n$. Suppose that the vertices of G are listed in arbitrary order as $v_1, v_2, \dots, v_n$.

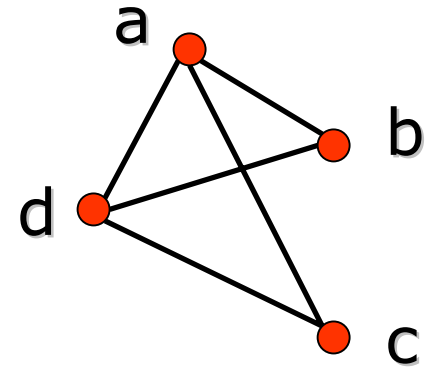
The **adjacency matrix** A (or A_G) of G , with respect to this listing of the vertices, is the $n \times n$ zero-one matrix with 1 as its (i, j) th entry when v_i and v_j are adjacent, and 0 otherwise.

In other words, for an adjacency matrix $A = [a_{ij}]$,

$$\begin{aligned} a_{ij} &= 1 && \text{if } \{v_i, v_j\} \text{ is an edge of } G, \\ a_{ij} &= 0 && \text{otherwise.} \end{aligned}$$

Representing Graphs

Example: What is the adjacency matrix A_G for the following graph G based on the order of vertices a, b, c, d ?



Solution:

$$A_G = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$$

Note: Adjacency matrices of undirected graphs are always symmetric (if $ab=0$ then $ba=0$)

Connectivity

Definition: A **path** of length $n-1$ from u to v , where n is a positive integer, in an **undirected graph** is a sequence of edges $e_1, e_2, \dots, e_n$ of the graph such that $e_1 = \{x_0, x_1\}$, $e_2 = \{x_1, x_2\}$, $\dots$, $e_n = \{x_{n-1}, x_n\}$, where $x_0 = u$ and $x_n = v$.

The path is a **circuit** if it begins and ends at the same vertex, that is, if $u = v$.

Connectivity

Definition (continued): The path or circuit is said to **pass through** or traverse $x_1, x_2, \dots, x_{n-1}$.

A path or circuit is **simple** if it does not contain the same edge more than once.

Connectivity

Let us now look at something new:

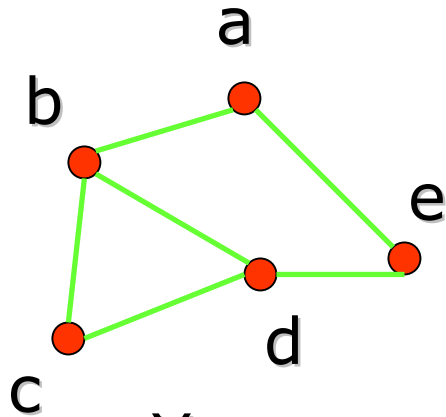
Definition: An undirected graph is called **connected** if there is a path between every pair of distinct vertices in the graph.

For example, any two computers in a network can communicate if and only if the graph of this network is connected.

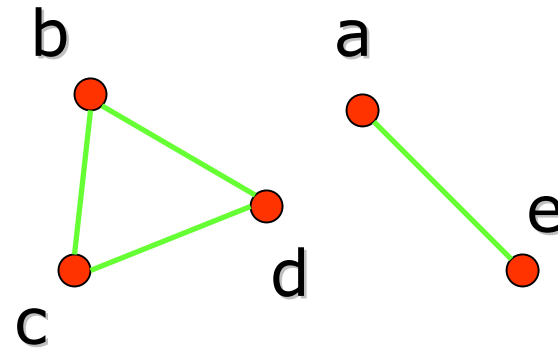
Note: A graph consisting of only one vertex is always connected, because it does not contain any pair of distinct vertices.

Connectivity

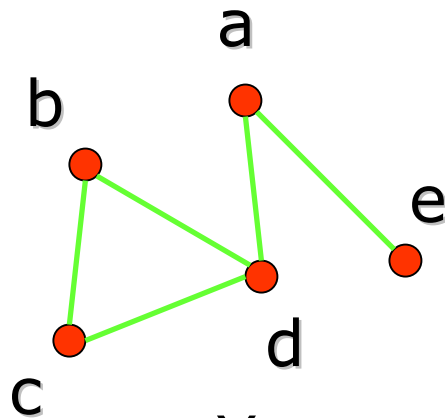
Example: Are the following graphs connected?



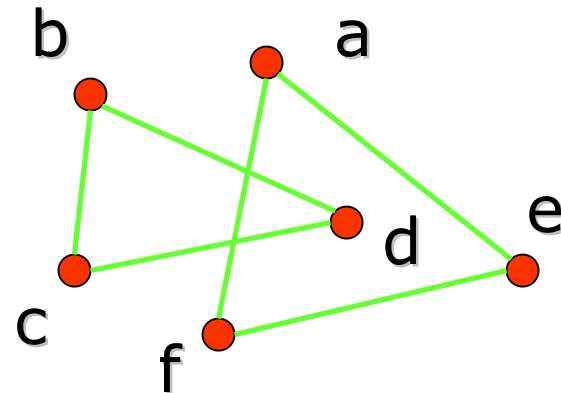
Yes.



No.



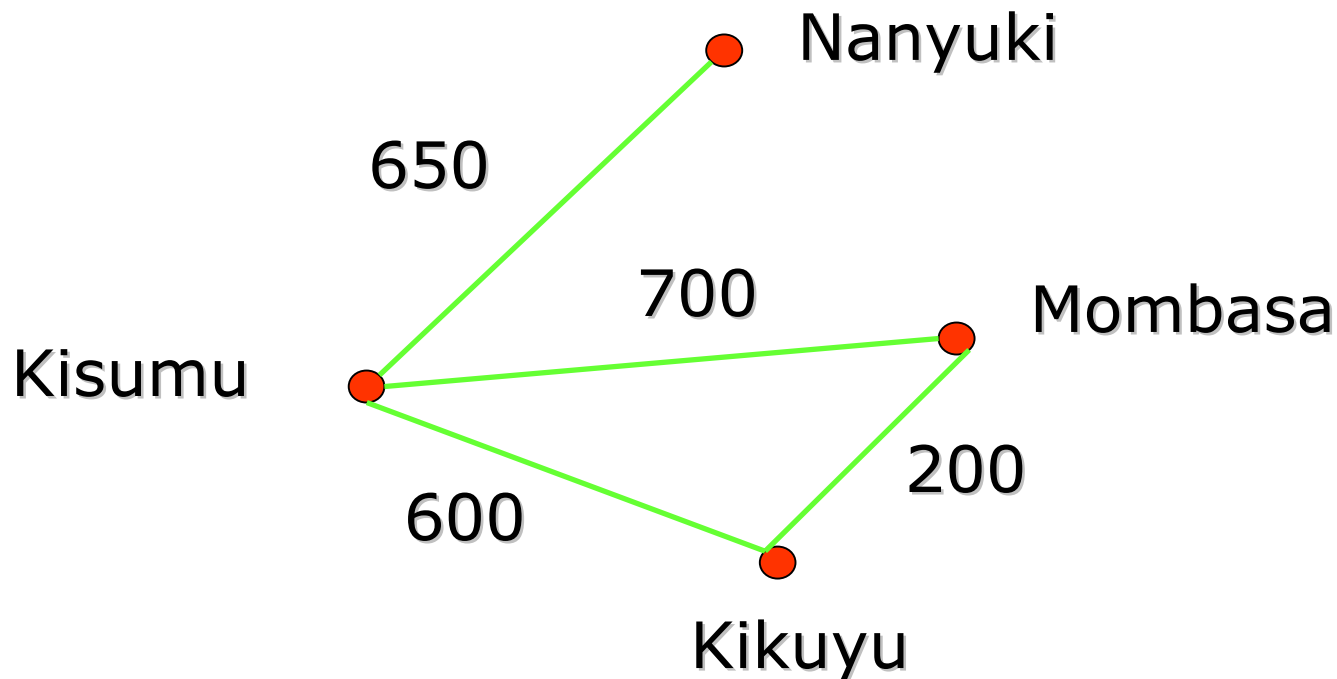
Yes.



No.

Shortest Path Problems

We can assign weights to the edges of graphs, for example to represent the distance between cities in a railway network:



Shortest Path Problems

Such weighted graphs can also be used to model computer networks with response times or costs as weights.

One of the most interesting questions that we can investigate with such graphs is:

What is the **shortest path** between two vertices in the graph, that is, the path with the **minimal sum of weights** along the way?

This corresponds to the shortest train connection or the fastest connection in a computer network.

Dijkstra's Algorithm

- Dijkstra's algorithm is an iterative procedure that finds the shortest path between a node known as the source and any other node in the graph
- It proceeds by finding the length of the shortest path from the source to successive vertices and adding these vertices to a distinguished set of vertices S .
- For the vertices in S , we already know the shortest path from the source
- The set $V-S$ contains the nodes in V that have not been added to S .
- We could find the shortest path from the source node to a destination node but this takes about the same time as finding the shortest path to all other nodes
- The problem solved is hence also referred to as single source shortest paths

Dijkstra's Algorithm

- The algorithm assumes we are given a directed graph $G=(V,E)$ where $V=\{1,2,\dots,n\}$ and vertex 1 is the source. C is a 2 dimensional array of costs, where $C[i,j]$ is the cost of moving from vertex i to j on arc $i \rightarrow j$. If arc $i \rightarrow j$ doesn't exist, assume $C[i,j] = \infty$ i.e its equal to infinity.
- $D[i]$ contains the cost of the current shortest path to vertex i .

Dijkstra's Algorithm

function Dijkstra

{

1. $S := 1;$

2. for $i := 2$ to n

3. $D[i] := C[i, j]$ //initialize D

4. For $i := 1$ to $n-1$

5. {

6. choose a vertex w in $V-S$ such that $D[w]$ is a minimum

7. add w to S

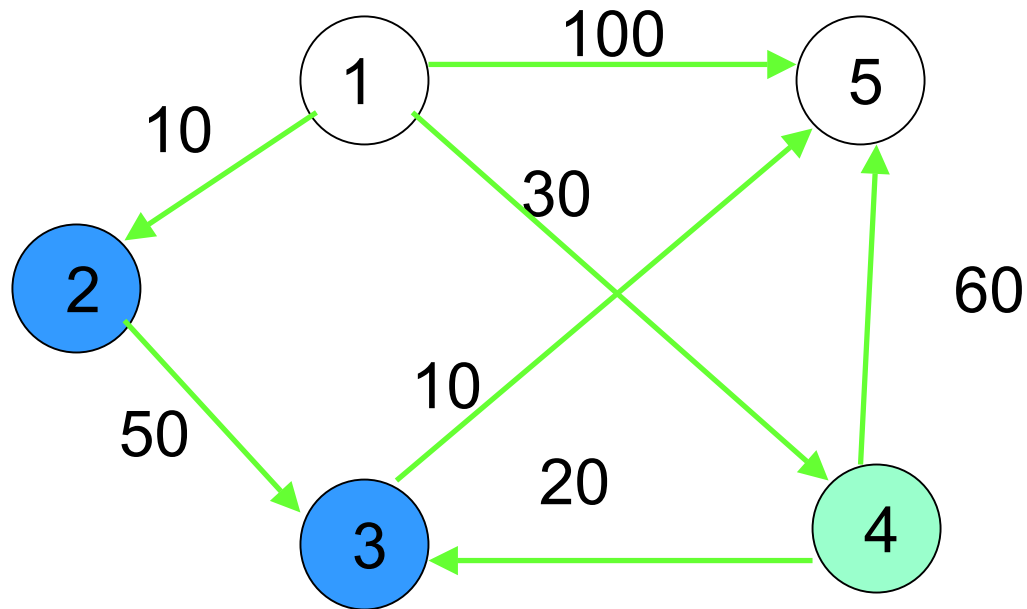
8. for each vertex v in $V-S$

9. $D[v] := \min(D[v], D[w] + C[w, v])$

10. } //end for

11. } //end function

Dijkstra's Algorithm



Given the above graph, use Dijkstra's algorithm to get the path to every other node from node 1?

Dijkstra's Algorithm

Using Dijkstra's algorithm, to get the path to a node, we have to have another array P of vertices such that $P[v]$ contains the vertex immediately before v in the shortest path

Initialize $P[v]$ to 1 for all $v \neq 1$. Every time we execute line 9 in the algorithm and $D[v] > D[w] + C[w, v]$ then we set $P[v] := w$.

The Traveling Salesman Problem

The traveling salesman problem is one of the classical problems in computer science.

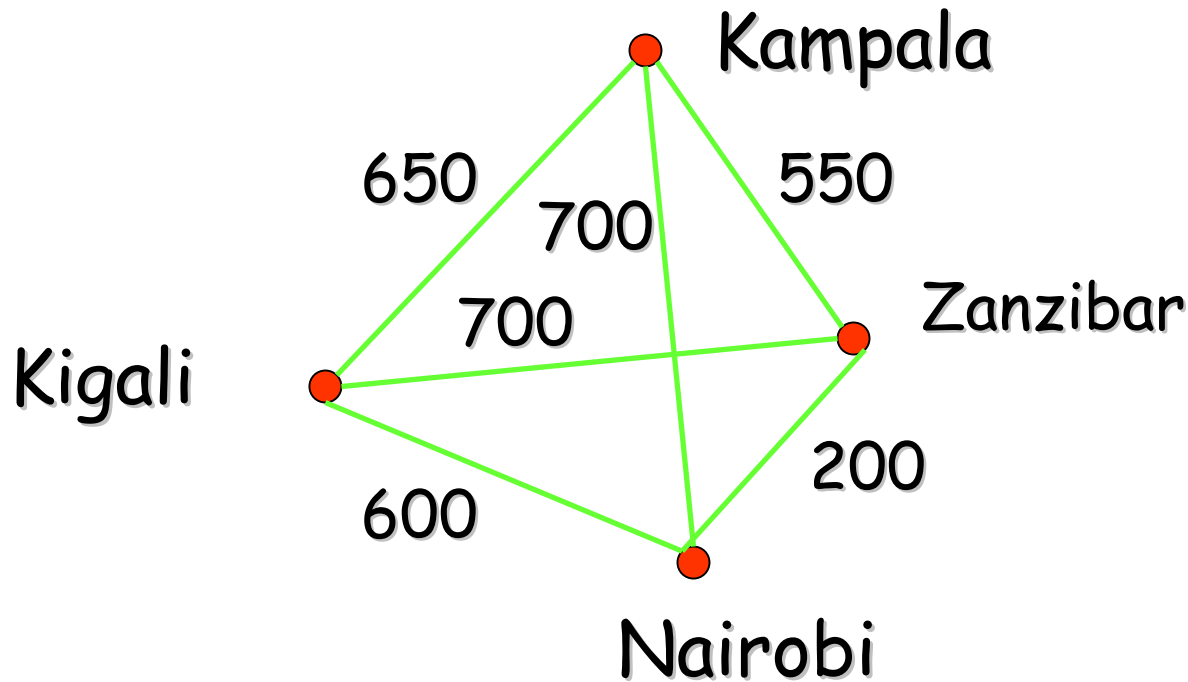
A traveling salesman wants to visit a number of cities and then return to his starting point. Of course he wants to save time and energy, so he wants to determine the **shortest path** for his trip.

We can represent the cities and the distances between them by a weighted, complete, undirected graph.

The problem then is to find the **circuit of minimum total weight that visits each vertex exactly once**.

The Traveling Salesman Problem

Example: What path would the traveling salesman take to visit the following cities?



Solution: The shortest path is Zanzibar, Nairobi, Kigali, Kampala, Zanzibar (2,000 miles).

The Traveling Salesman Problem

Unfortunately, no algorithm solving the traveling salesman problem acceptable time complexity has been devised yet.

This means that for large numbers of vertices, solving the traveling salesman problem is impractical.

In these cases, we can use efficient **approximation algorithms** that determine a path whose length may be slightly larger than the traveling salesman's path.

The
End